Same Parts, Different Arrangements Matched Edits for Budgeted Anomaly Inspection

Anonymous CVPR submission

Abstract

An anomaly detector that raises an alarm on a faulty product is useful only if it also tells the inspector where to look. We show that this second step can fail even when detection is perfect. We study a tray that must hold one pushpin per compartment and edit real photographs so that the same pin moves either within its own compartment, which is allowed, or into a neighbouring compartment, which breaks the rule. Because both edits change exactly the same pixels, they reveal precisely which two compartments need checking. A recent detector flags every faulty tray, yet to find nearly all the misplaced pins it also sends the inspector to 11 of the 13 untouched compartments. The cause lies in how the detector is trained: it learns to mark every pin as anomalous whenever one pin is added. Teaching it to mark only the pin that was actually moved reduces the extra checks to less than half a compartment, while every fault is still detected. Matched valid and invalid edits thus turn anomaly detection benchmarks into a measure of the work an alarm creates.

1. Introduction

Visual anomaly detection is usually assessed by whether a system flags defective products [3, 4, 16], yet on a production line an alarm is only the first step: an operator must then locate the defect. For *logical* anomalies [5] this second step is particularly demanding. A pushpin tray with one empty and one doubled compartment contains the correct number of intact pins, and only their arrangement is wrong; a detector is helpful if its map points to the two affected compartments, and of little use if it highlights the other thirteen as well. Current benchmarks do not measure this cost. MVTec LOCO [5] reports image-level AUROC and saturated per-region overlap, and neither states how many locations an inspector must examine to recover nearly every problem. Measuring it requires knowing exactly which locations are wrong, which natural logical defects rarely provide; an extra screw in a bag may be any of several identical screws [6].

We obtain exact targets by intervention (Fig. 1a). From each held-out normal photograph we move an intact pin either within its compartment (a *valid sham*) or by the same distance into an occupied neighbour (a *fault*). Both edits

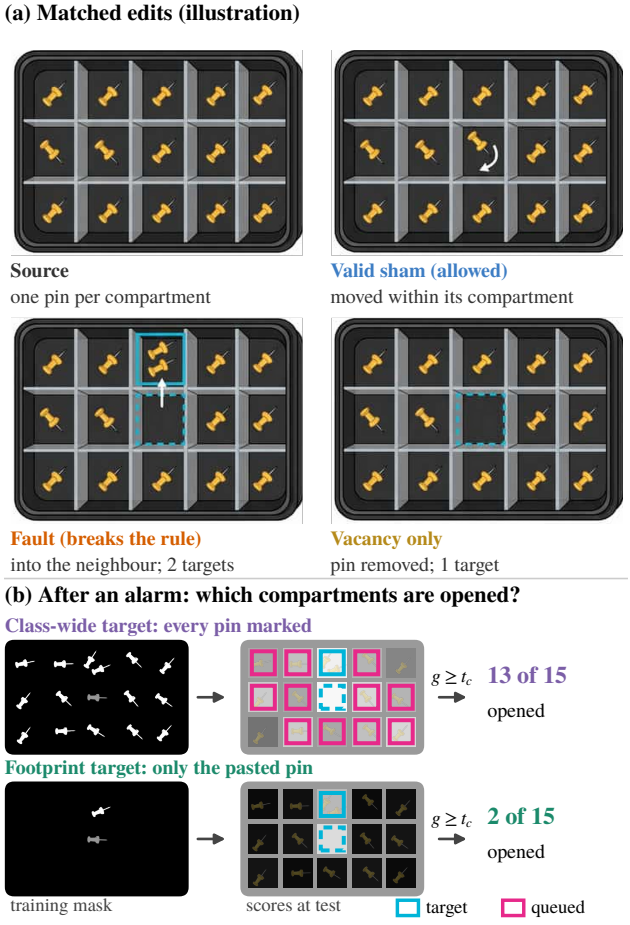


Figure 1. Where should an inspector look after an alarm? (a) Matched edits of one tray (rule: one pin per compartment): the same pin moves the same distance within its compartment (valid, no target) or into an occupied neighbour (fault). Cyan: compartments to check (dashed: vacated; solid: recipient). (b) Schematic: marking every pin of the inserted class in training makes nearly the whole tray score high, so 13 of 15 compartments are opened; marking only the pasted pin opens just the 2 targets. Measured at 95% recall: 11.07 vs. 0.20–0.45 untouched compartments (Tab. 1). Tray art created in BioRender.

share the cutout, fill and displacement, so any difference in the detector's response is caused by the rule violation, and the fault fixes the two compartments that must be inspected. A scene alarm then decides whether a tray is opened, and a compartment cutoff fixed on development data decides which compartments are queued; we report the fraction of target compartments found (recall) and the number of untouched compartments queued (cost).

A toy example makes the failure concrete (Fig. 1b). Suppose a detector assigns each compartment a score, and the inspector opens every compartment whose score exceeds a cutoff chosen so that 95% of misplaced pins are found. If the detector was trained to call every pin anomalous once a pin is inserted, the untouched compartments score almost as high as the two affected ones, the cutoff must fall below them, and the inspector opens 13 of 15 compartments to find 2. If it was trained to mark only the inserted pin, the same cutoff separates the two affected compartments from the rest, and the inspector opens 2. Both detectors raise the same alarm and would receive the same image-level AUROC; only the second is useful.

We apply this protocol to SALAD [6], a recent logical anomaly detector whose composition branch scores predicted part maps, and ask two questions: how much inspection effort does a detector that catches every fault still impose, and what in its training determines that effort? Our paper offers the following contributions. First, we introduce matched valid and invalid edits with identical pixel changes, together with a recall–cost criterion calibrated on development data (Sec. 3). Second, we show that the released composition branch detects all 138 faults and ranks each above its sham, yet queues 11.07 of 13 untouched compartments per fault at 95% target recall (Sec. 5.1). Third, we show that the training target governs this cost: marking only the pasted part reduces it to 0.20–0.45 compartments across two seeds, with paired intervals far from zero (Sec. 5.2). Finally, the same design identifies which SALAD branch carries the evidence under a second rule and isolates a degree pathway in a masked link predictor (Secs. 6 and 7).

2. Related work

Industrial anomaly detection was first benchmarked on local structural defects such as scratches and dents [4], where patch-feature memory banks [16] and student–teacher networks [3] perform strongly. MVTec LOCO [5] adds logical defects that violate constraints on the number, position or combination of parts and evaluates localization with the saturated per-region overlap (sPRO), which tolerates ambiguous ground truth. Component-based methods segment parts and compare attributes, correspondences or executable rules [9, 13, 15, 21], and SALAD [6] combines a composition branch on part maps with local and global appearance branches. Many of these detectors train a discriminative head on synthetic defects: CutPaste [11] pastes image patches, DRAEM [19] blends Perlin-masked textures and segments the altered region, LogicAL [22] synthesizes logical defects, and SALAD inserts and relabels parts in composition maps. This line of work establishes strong detection and region overlap, but it neither measures how many locations an inspector must open once an alarm fires nor examines how the target mask of the synthetic edits shapes that number. We hold the detector and its edits fixed and study exactly these two questions.

Our evaluation borrows from matched and counterfactual testing outside anomaly detection. Contrast sets [7] and counterfactually augmented data [10] probe language models with minimally different inputs that carry different labels, and Winoground [18] pairs captions that share words but differ in their relations; such pairs reveal shortcut features that ordinary test sets reward [8]. Our valid shams play this role for anomaly maps, since they alter as many pixels as the fault while respecting the rule. Operating points follow split-conformal calibration on normal photographs [1, 2], which bounds the false-alarm rate, and for graphs we build on the known effects of target-link masking and node degree in link prediction [20, 23], measuring how masking changes the available signal when two graphs share the same degree sequence.

3. Method

3.1. Problem setting

A photograph x_i shows a tray of $|\mathcal{E}| = 15$ compartments arranged in a 5×3 grid, and the published rule requires exactly one pin per compartment [5]. A part extractor maps the photograph to a six-class part map $m_i \in \{0,\dots,5\}^{256\times256}$, and the composition branch f of the detector returns a score map $S_i = f(m_i) \in [0,1]^{256\times256}$ in which higher values indicate anomalies (Fig. 2). For a compartment $e \in \mathcal{E}$, $S_i|_e$ denotes the scores of its pixels; compartment boundaries are registered once from the released label maps.

3.2. Matched edits

Using the released class mask and a two-pixel RGB halo, we cut out one intact pin and translate it without resampling, which yields four variants per photograph (Fig. 1a, Fig. 3). The *source* is the unedited photograph. In the *valid sham*, the pin moves within its own compartment, so every compartment still holds one pin. In the *fault*, the pin moves by the same squared displacement into a face-adjacent occupied compartment, leaving its source compartment a_i empty and the recipient b_i with two pins. The *vacancy-only* variant removes the pin without pasting it elsewhere. The sham and the fault share the cutout, the fill (the median of the local dark background) and the paste area, and therefore change exactly the same number of RGB pixels; collisions,

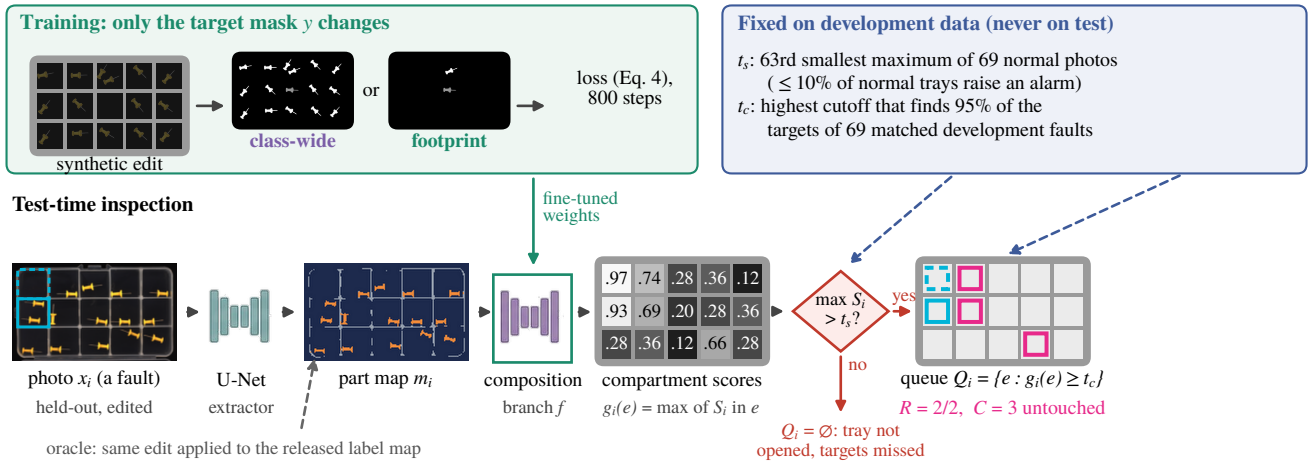


Figure 2. Only the training target changes; everything at test time is fixed in advance. Training (green) differs only in the target mask. At test, a part map comes from a U-Net (*extracted*) or from the edited label map (*oracle*); the scene alarm (t_s) and the compartment cutoff (t_c) are fixed on development data (blue). The values are illustrative and match the example under Eq. (4) ($R = 2/2$, $C = 3$). Network icons created in BioRender.

divider clearance and background agreement are verified before any score is computed. Displacements range from 19 to 51 pixels.

Each variant has a target set $\mathcal{T}_i$ of compartments to inspect: $\mathcal{T}_i = \{a_i, b_i\}$ for a fault, $\mathcal{T}_i = \{a_i\}$ for a vacancy, and $\mathcal{T}_i = \emptyset$ for a source or a sham, where every queued compartment is a false mark. Since the vacancy-only edit retains the empty half of the fault, comparing the two isolates the effect of the additional pin.

Every variant is scored through two interfaces with identical detector weights. The *oracle* interface moves the pin directly in the released label map, while the *extracted* interface re-segments every RGB variant, including the source, with an independent U-Net trained only on normal training photographs (97.3% development pin IoU). We report the extracted interface, which a deployed system would use, unless stated otherwise.

3.3. Calibrated inspection queue

We model the two decisions of an inspector, whether to open a tray and which compartments to check, with two cutoffs that are fixed before testing (Algorithm 1): a scene cutoff $t_s \in [0, 1]$ on the highest pixel score of a map, and a compartment cutoff $t_c \in [0, 1]$ on the highest score inside a compartment. Writing p for a pixel of the 256×256 grid, a tray raises an alarm when its highest pixel score exceeds the scene cutoff,

$$\text{alarm}_i = \mathbb{1}\big[\max_p S_i(p) > t_s\big], \tag{1}$$

where $\mathbb{1}[\cdot]$ equals 1 if its argument holds and 0 otherwise. Let $M_{(1)} \le \cdots \le M_{(69)}$ be the sorted maxima $\max_p S_v(p)$ of 69 unedited validation photographs v. We set

$$t_s = M_{(63)}, \qquad 63 = \lceil 0.9\,(69+1) \rceil, \tag{2}$$

a strict split-conformal cutoff [2] with the following guarantee.

Proposition 1 (Normal alarm rate) *Let M be the maximum score of a new normal tray. If M and the 69 validation maxima are exchangeable and almost surely distinct, then* $\Pr(M > t_s) = 1 - 63/70 = 0.1$.

Proof. By exchangeability the rank r of M among the 70 values is uniform, and $M > M_{(63)}$ exactly when $r \ge 64$, with probability $7/70$; ties only lower it. □

Each compartment then receives its maximum score $g_i(e) = \max S_i|_e$, and the inspection queue at compartment cutoff t_c is

$$\mathcal{Q}_i = \begin{cases} \{e \in \mathcal{E} :\ g_i(e) \ge t_c\} & \text{if alarm}_i = 1, \\ \emptyset & \text{otherwise,} \end{cases} \tag{3}$$

so that the targets of a tray without an alarm count as missed. Over n faults, the target recall R and the inspection cost C are

$$R = \frac{1}{2n}\sum_{i=1}^{n} |\mathcal{Q}_i \cap \mathcal{T}_i|, \qquad C = \frac{1}{n}\sum_{i=1}^{n} |\mathcal{Q}_i \setminus \mathcal{T}_i|, \tag{4}$$

i.e., the fraction of target compartments that are queued and the mean number of untouched compartments queued per fault, which ranges from 0 to 13. For example, a fault whose queue contains both targets and three other compartments contributes $2/2$ to recall and 3 to cost.

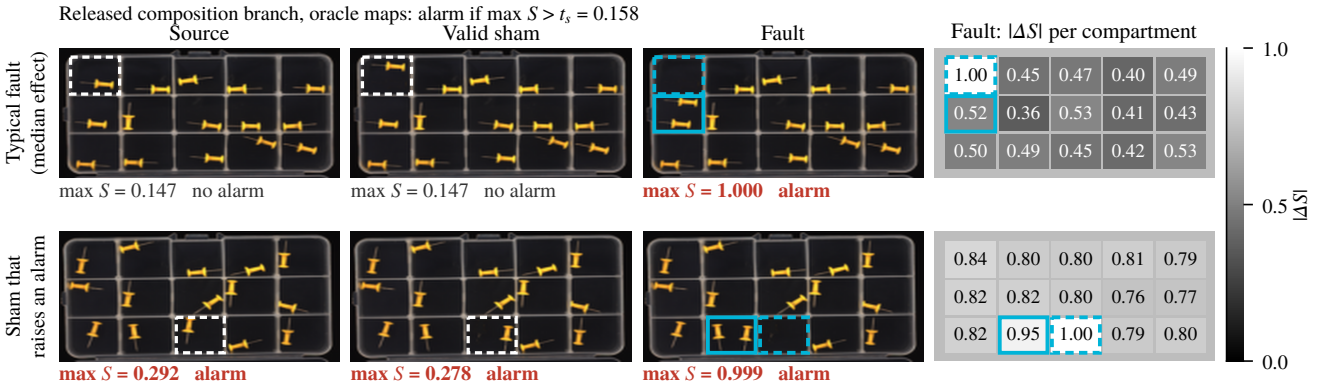


Figure 3. Adding a pin changes the released map across the whole tray (real matched edits, oracle maps). Top: the photo whose released fault-minus-sham score difference is the median. Bottom: the first sham that raises an alarm; its source does too. White dashed: compartment of the moved pin; cyan: fault targets. Right: per-compartment change from source to fault, $|S^{\text{fault}} - S^{\text{source}}|$; in the bottom row every compartment changes. Photos: MVTec LOCO AD, CC BY-NC-SA 4.0.

Algorithm 1 Calibrated inspection queue

Require: map model f; cells $\mathcal{E}$; normal validation photos $\mathcal{V}$; development faults $\mathcal{D}$ with targets $\mathcal{T}_j$

$t_s \leftarrow$ 63rd smallest of $\{\max_p f(m_v)(p)\}_{v\in\mathcal{V}}$ ▷ $|\mathcal{V}| = 69$

$G \leftarrow \{g_j(e) : j \in \mathcal{D},\ e \in \mathcal{T}_j\}$, with $g_j(e) = 0$ if j has no alarm

$t_c \leftarrow$ the $\lceil 0.95\,|G| \rceil$-th largest value of G ▷ 95% cutoff

for each test photo i **do**

  $S_i \leftarrow f(m_i)$; **if** $\max_p S_i(p) \le t_s$ **then** $\mathcal{Q}_i \leftarrow \emptyset$

  else $\mathcal{Q}_i \leftarrow \{e \in \mathcal{E} : \max S_i|_e \ge t_c\}$

return R and C from Eq. (4)

The compartment cutoff is fitted on 69 matched *development* faults $\mathcal{D}$, constructed in the same way from validation photographs, and never on test data. Let $G = \{g_j(e) : j \in \mathcal{D},\ e \in \mathcal{T}_j\}$ be the multiset of the $2 \times 69 = 138$ development target scores, with $g_j(e) = 0$ for a fault without an alarm, and let $G^{(k)}$ denote its k-th largest element. The *95% cutoff* is the highest threshold that attains 95% target recall on these faults,

$$t_c^{95} = G^{(\lceil 0.95\,|G| \rceil)} = G^{(132)}. \tag{5}$$

A *fixed-cost cutoff* instead admits at most 0.5 untouched compartments per development fault and is the 35th largest untouched score, since $\lfloor 0.5 \cdot 69 \rfloor + 1 = 35$; for this cutoff the comparison in Eq. (3) is strict ($g_i(e) > t_c$). Both cutoffs are fitted separately for each map and interface.

3.4. Footprint supervision

SALAD trains its composition branch on synthetic edits of normal part maps. For an edited map, a discriminative U-Net predicts a logit $z_p \in \mathbb{R}$ at every pixel p of the pixel set P, so that $\sigma(z_p) = 1/(1 + e^{-z_p})$ is the predicted anomaly probability, and a binary target mask $y \in \{0,1\}^P$ states which pixels should be called anomalous. With the sigmoid focal loss $\mathrm{FL}(z, y) = -\alpha_y (1-q)^\gamma \log q$, where $q = \sigma(z)$ if $y = 1$ and $q = 1 - \sigma(z)$ otherwise, $\alpha_1 = \alpha = 0.25$, $\alpha_0 = 1 - \alpha$ and $\gamma = 2$ [12], the branch minimizes

$$\mathcal{L} = 5 \cdot \frac{1}{|P|} \sum_p \mathrm{FL}(z_p, y_p) + \frac{1}{|P|} \sum_p \big|\sigma(z_p) - y_p\big|, \tag{6}$$

where the first term emphasizes hard pixels, the second penalizes the absolute probability error, and the weight 5 is that of the released implementation. Since y is the only supervision of the map, it determines which pixels the branch learns to score high. When a part of class k is inserted, producing the edited map m', the released implementation marks every pixel of that class [6],

$$y^{\text{cw}}(p) = \mathbb{1}[m'(p) = k] \qquad \text{(class-wide)}, \tag{7}$$

reflecting the view that an extra screw in a bag may be any of the screws. In a tray, however, only one compartment is over-full, which motivates marking only the footprint F of the pasted donor,

$$y^{\text{fp}}(p) = \mathbb{1}[p \in F] \qquad \text{(footprint)}. \tag{8}$$

For relocations, both policies additionally mark the vacated footprint, shams always receive $y \equiv 0$, and same-image relabelling targets are left unchanged, so the two policies differ only in the insertion part of the mask. In seed 1, a relocation marks on average 3,823 pixels under the class-wide policy and 461 under the footprint policy. The rule is used only to construct training edits; at test time the model observes a part map alone.

4. Experimental setup

4.1. Datasets

The main experiments use the pushpin category of MVTec LOCO AD [5]. Its 372 normal training photographs train the part extractor and supply the rule-aware training edits. The 69 normal validation photographs set the scene cutoff and yield the 69 matched development faults that set the compartment cutoff. All 138 normal test photographs, used neither for training nor for calibration, yield the matched test edits of Sec. 3.2. For the second rule we use 89 of the 94 eligible normal test photographs of the juice-bottle category (Sec. 6), and the graph analysis uses 78 GlaS histology fields from 12 donors [17] and cardiac structure graphs from HVSMR-2.0 [14] (Sec. 7).

4.2. Experimental settings

Each fine-tune starts from the released SALAD composition weights with the reconstruction autoencoder frozen. A batch contains two clean maps, one SALAD-style synthetic edit and one rule-aware edit (a relocation, sham or vacancy) built from the training photographs. We train for 800 Adam steps and keep the final checkpoint, using two seeds. Within a seed, the two policies receive identical inputs, edits, optimization steps, loss weights and pixel weights, and differ only in y. Further details are in the supplement.

4.3. Baselines

The reference is the released SALAD composition branch, read out per compartment by its maximum or 99th-percentile score. The pre-specified control is the *class-wide* fine-tune, which follows exactly the recipe above with $y = y^{\mathrm{cw}}$, so that the comparison with the *footprint* fine-tune isolates the target. An upstream-only continuation, which uses SALAD's original edit families without rule-aware edits, separates the target from the rule-aware training data (Sec. 5.3). For the branch analysis we compare the local RGB, global RGB and fused scores of the full released SALAD with an explicit one-pin-per-compartment rule applied to the extracted maps. We do not compare other logical detectors [13, 21]: our question is how the training target of one architecture sets the inspection cost, and changing the architecture would confound the two.

4.4. Evaluation metrics

We report target recall R and inspection cost C (Eq. (4)) at the development cutoffs, together with alarm rates on all four variants. The primary comparison is the paired difference in cost between the footprint and class-wide policies on extracted maps,

$$\Delta C = \frac{1}{n}\sum_{i=1}^{n}\Big(|\mathcal{Q}_i^{\mathrm{fp}} \setminus \mathcal{T}_i| - |\mathcal{Q}_i^{\mathrm{cw}} \setminus \mathcal{T}_i|\Big), \tag{9}$$

where $\mathcal{Q}_i^{\mathrm{fp}}$ and $\mathcal{Q}_i^{\mathrm{cw}}$ are the queues of the same fault under each policy at its own 95% cutoff; negative values favour the footprint policy. We write a paired difference as "$+x$ points $[a, b]$": x is the mean difference between two maps on the same photographs, in percentage points (pp) for rates or in compartments for cost, and $[a, b]$ is its 95% bootstrap interval from 4,000 resamples of the original photographs, in which all variants, policies and interfaces of a photograph are resampled together. Intervals are conditional on the fitted models and cutoffs.

Before testing, we specified that a useful reduction must remove at least one untouched compartment with an interval excluding zero, while keeping the achieved recall at or above 90% and within 5 points of the control, the fault and vacancy alarm rates at or above 95% and within 3 points, and the increase in sham alarms below 5 points. Attributing the effect to the target additionally requires that the class-wide control meet a *competence criterion* on development data: alarm rates and compartment AUROC of at least 90%. Two seeds and two fixed halves of the test photographs (63 and 75) assess stability. Because some systems present a fixed number of compartments, we also report the fraction of faults whose two highest-scoring compartments are both targets (*top 2*) and the mean rank of the second target, breaking ties by compartment index. Finally, as a counterpart to sPRO [5, 6], we sweep a common threshold over all compartments, with targets as positives and untouched fault compartments together with all source and sham compartments as negatives, and integrate target recall up to a false-compartment rate of 0.05, normalized by 0.05 (*region area*). This summary is computed on test data and ignores the scene alarm, so it measures ranking quality rather than the cost at an operating point.

5. Results

5.1. Diagnosis of the released model

The released composition branch raises an alarm on all 138 faults and all 138 vacancies under both interfaces, and scores every fault above its matched sham (138/138; exact 95% interval 97.4–100%). On oracle maps, the AUROC of the affected compartments rises from 48.6% on the source, which is chance level, to 98.2% after the fault. The ranking is also accurate: on extracted maps, both targets are among the two highest-scoring compartments for 77.5% of faults and among the three highest for 94.2%, and the second target appears at rank 2.43 on average. Scoring compartments by their 99th percentile instead of their maximum raises the top-2 rate to 87.0% (+9.4 pp [4.3, 14.5]) with the same alarm and weights.

Good ranking does not yield a short queue. At the 95% cutoff, the released map recovers 95.7% of the target compartments but queues 11.07 of the 13 untouched compart-

Table 1. Pushpin inspection on extracted maps (138 test photographs). Every row raises an alarm on all faults and vacancies. Sham: sham alarms; 2nd: mean rank of the second target; R (%) and C at the 95% development cutoff; Area: region area (%). Metrics are defined in Sec. 4.4; oracle maps are in supplement Tab. 2.

Map, compartment score	Sham	Top 2	2nd	R (%)	C	Area
Released, max	7	77.5	2.43	95.7	11.07	81.6
Released, 99th pct.	7	87.0	2.32	96.0	11.41	77.3
Class-wide, seed 1	19	0.0	15.00	97.1	13.00	39.3
Footprint, seed 1	9	87.7	2.13	95.3	**0.20**	**96.3**
Class-wide, seed 2	22	0.0	15.00	96.4	13.00	39.4
Footprint, seed 2	20	74.6	2.28	96.0	**0.45**	89.6

ments per fault (Tab. 1), so that the inspector examines nearly the entire tray. The 99th-percentile readout, although better at rank two, is no better here (11.41 at 96.0%), and the 98% cutoff raises the cost to 12.56. Conversely, when the cost is held to 0.30 compartments by the fixed-cost cutoff, the released map recovers only 81.9% of the targets (Fig. 4d).

The reason is that the fault alters the released map across almost the entire tray (Fig. 5). On oracle maps, 90.3% of untouched compartments contain a pixel whose score changes by at least 0.5, although only 5.4% of their pixels change by that amount, indicating sharp, isolated peaks rather than broad regions. The vacancy-only edit, which leaves the same empty compartment, affects only 2.9% of the untouched compartments and the sham 0.7%. The fault-minus-vacancy difference is 87.4 points [82.4, 91.9] on oracle maps and 86.6 [81.4, 91.2] after extraction. The additional pin rather than the vacancy thus causes the widespread response, consistent with a class-wide target that labels every pin as anomalous once a pin is inserted.

5.2. Effect of footprint supervision

Fine-tuning with footprint targets reduces the cost at 95% recall from 11.07 untouched compartments to 0.20 and 0.45 in the two seeds, at an achieved recall of 95.3% and 96.0% (Tab. 1). The paired reductions relative to the released model are 10.88 [10.12, 11.56] and 10.62 [9.86, 11.31] compartments, and relative to the class-wide fine-tunes, the pre-specified control, 12.80 [12.72, 12.87] and 12.55 [12.45, 12.65] (Fig. 6). Every fault and vacancy still raises an alarm, and all recall differences have intervals that include zero and remain above the 5-point floor. The reduction is also observed on oracle maps (10.30 and 10.04 relative to the released model) and on both fixed halves of the photographs (-12.87 and -12.75 for seed 1, -12.63 and -12.48 for seed 2). Figure 4 illustrates a typical case in which the released map queues all 13 untouched compartments and the footprint map queues none.

At the fixed-cost cutoff, the footprint heads recover 100.0% and 98.2% of targets at achieved costs of 0.38 and

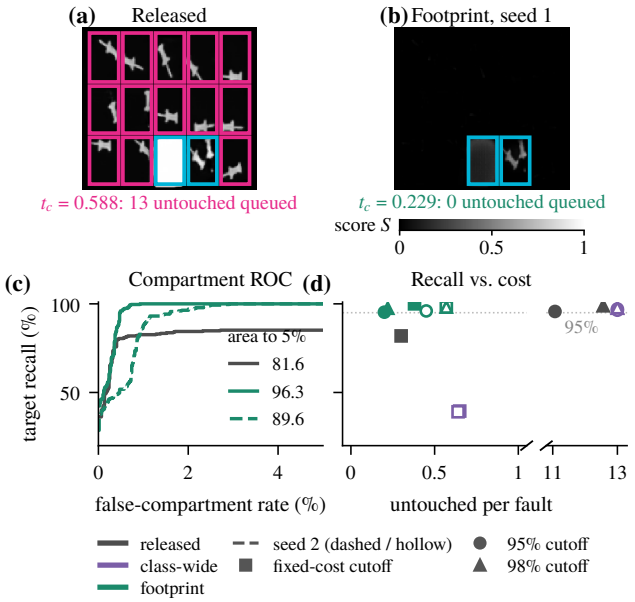


Figure 4. Footprint supervision empties the queue. (a,b) Extracted maps of the median fault (greyscale); cyan: targets, pink: untouched compartments at or above the pre-specified 95% cutoff t_c. (c) Compartment ROC from a test sweep, with the area up to 5% from Tab. 1 (class-wide curves, area 39.3–39.4, omitted). (d) Recall against cost at the fixed-cost (square), 95% (circle) and 98% (triangle) development cutoffs; hollow: seed 2; the two class-wide seeds overlap at fixed cost; the axis is broken between 1 and 11.

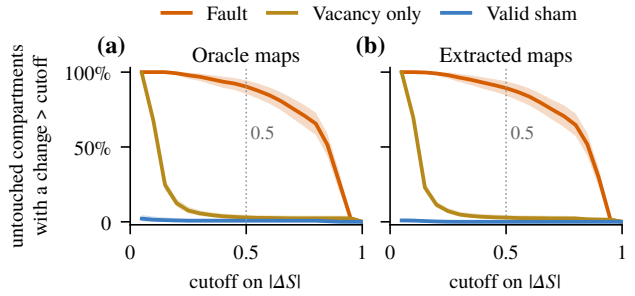


Figure 5. An extra pin, unlike a vacancy or a valid move, changes the released map across the tray. Share of the 13 untouched compartments with a pixel whose score changes by more than the cutoff, with 95% intervals over photos.

0.57, compared with 81.9% at 0.30 for the released model. In a descriptive sweep at an equal realized cost of 0.50 compartments, recall is 82.6% for the released model and 100.0% and 97.1% for the footprint heads, corresponding to gains of $+17.4$ [13.4, 21.0] and $+14.5$ [9.8, 18.8] points. The region area rises from 81.6% to 96.3% and 89.6%, in the same direction.

Two secondary outcomes depend on the seed. Sham alarms increase from 7 for the released model to 9 for seed 1

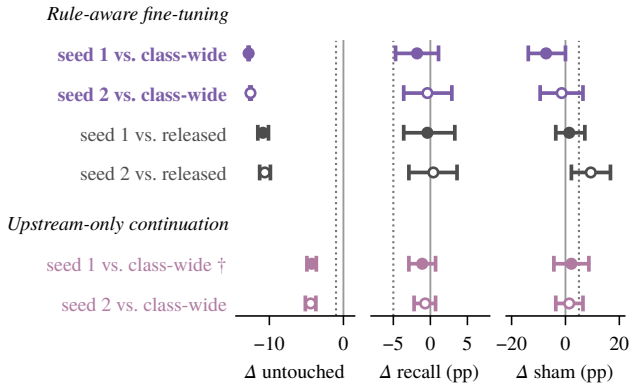


Figure 6. Footprint targets remove 10 to 13 untouched compartments per fault without losing recall. Paired differences on extracted maps, footprint minus each comparator, at the pre-specified cutoffs, with 95% intervals; hollow: seed 2; bold: the primary contrast. Dotted: pre-specified thresholds (-1 compartment, -5 and $+5$ pp). †: this seed's control fails the competence criterion (Sec. 5.3). Oracle maps and values: supplement Tabs. 3 and 8.

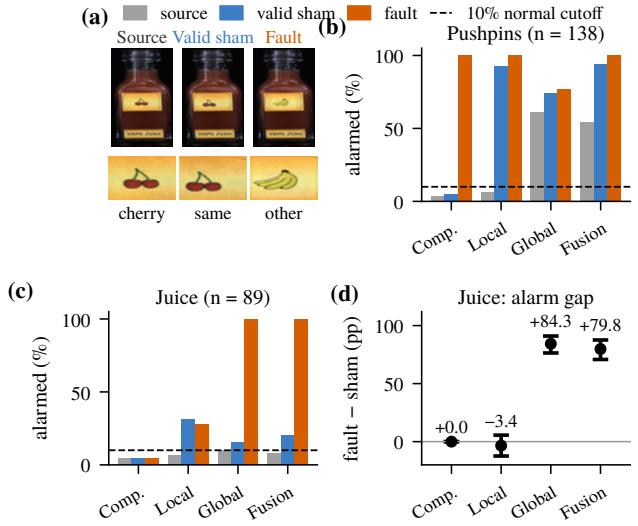


Figure 7. Which SALAD branch separates a fault from a valid edit? (a) Juice edit: a same-type (valid) or other-type (fault) icon, pasted identically. (b,c) Scenes alarmed per branch at its own 10% normal cutoff (dashed); on pushpin composites, global RGB and fusion also alarm on most untouched sources. (d) Juice fault-minus-sham alarm gap, 95% intervals. Photos: MVTec LOCO AD, CC BY-NC-SA 4.0.

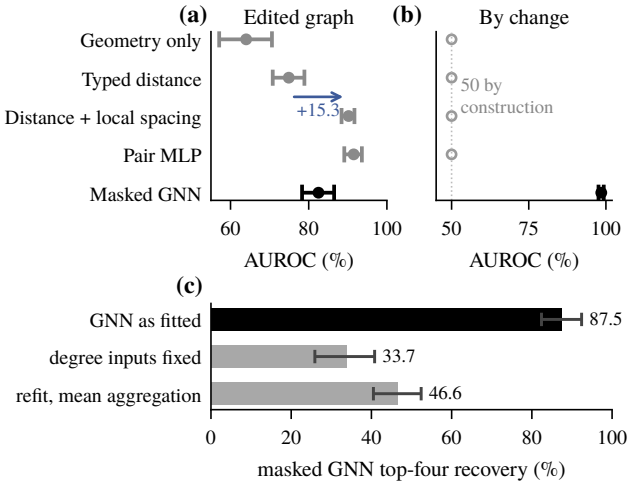


Figure 8. A masked GNN responds to switched links through node degree (78 GlaS fields, 12 donors). (a) Edited-graph AUROC; local spacing adds 15.3 points to typed distance. (b) AUROC of the change between matched graphs; hollow: 50 by construction. (c) Top-four recovery of the masked GNN as fitted, with degree inputs fixed, and refit with mean aggregation. Donor-paired 95% intervals.

(+1.4 points [−3.6, 7.2]) but to 20 for seed 2 (+9.4 [2.2, 16.7]); relative to the class-wide control, which is the pre-specified comparison, neither seed exceeds the 5-point margin (−7.2 and −1.4). The top-2 rate rises to 87.7% for seed 1 but falls to 74.6% for seed 2, below the released model.

The class-wide fine-tunes, in contrast, retain every alarm but assign high scores to the entire tray: all 13 untouched compartments are queued and the top-2 rate is zero. With a development compartment AUROC of about 48%, they fail the competence criterion of Sec. 4.4. Within this recipe, the comparison with the class-wide arm attributes the difference to the target alone, whereas the comparison with the released model measures the benefit of the complete adaptation, which also introduces rule-aware edits.

5.3. Continuation of the original recipe

To separate the effect of the target from that of the rule-aware edits, we also continue training with SALAD's original synthetic edit families only, replacing the rule-aware slot with a clean map. The class-wide head of seed 2 passes the competence criterion (97.3% development AUROC) and reproduces the behaviour of the released model, with 88.8% of untouched compartments changing by at least 0.5. Footprint targets reduce this fraction to 1.3% and the cost from 11.50 to 7.07, a paired reduction of 4.43 [3.72, 5.14] at 96.0% and 95.3% recall. Seed 1 shows a similar reduction (4.28), but its control does not reach the AUROC criterion. Footprint targets thus shorten the queue even without rule-aware training; rule-aware edits give the lowest costs.

6. Branch selection under two rules

SALAD fuses three branches: a local RGB appearance map, the composition map studied above, and a global RGB score

that pools student features over the scene and over regions defined by the part map [6]. Our replay of the complete released system reproduces the published 100.0% logical AUROC on 91 native logical pushpin faults and 138 normal photographs.

On the matched pushpins, however, the shams reveal a cost that this AUROC does not capture (Fig. 7b). Fusion ranks every fault above its sham but raises an alarm on 130 of 138 extracted shams and 75 of 138 sources, with 55 shams alarming although their source does not. The local RGB branch alone alarms on 128 shams and only 9 sources; since both arms contain the same compositing, it responds to the edit rather than to the rule. The composition branch alarms on 7 shams, and an explicit one-pin-per-compartment rule applied to the same extracted maps raises no sham alarm while detecting every fault.

A second rule shows that the informative branch depends on the rule. MVTec LOCO requires the icon on the upper label of a juice bottle to match its liquid [5]. We paste intact icons from a disjoint bank of training photographs into the same label region (Fig. 7a); an icon of the same type yields a valid sham and an icon of another type a fault. Both use the same offset, blending and footprint, and their mean blended RGB changes differ by at most half a grey level. Since the liquid, the bottle and the part map are unchanged, the composition branch returns identical maps for all three variants. The global RGB branch ranks all 89 faults above their shams and alarms on every fault but only 14 shams, a fault-minus-sham alarm gap of $+84.3$ points [76.4, 91.0] (Fig. 7d), slightly above that of fusion ($+79.8$ [70.8, 87.6]). The local RGB branch ranks the upper label first among five bottle regions for 74.2% of faults but also for 85.4% of shams, and alarms on 25 faults and 28 shams. The two rules therefore favour different branches: composition evidence decides placement in a tray, whereas global appearance captures a label–liquid correspondence that the part map cannot express.

7. Response evidence in part graphs

The matched design also applies to graphs of parts. On the GlaS gland graphs and HVSMR cardiac graphs (Sec. 4) we apply *degree-preserving switches* that remove two links and add two others so that every node keeps its degree. Readouts are evaluated by how well they rank the four switched pairs. On GlaS, adding local gland spacing to typed centroid distance improves the edited-graph AUROC by 15.3 points [10.9, 20.1], and the gain persists for additional length-matched switches and under joint matching of raw and local distances ($+16.8$ [11.3, 23.0]); these tiers reuse donors and use no pathology labels.

For a candidate link $e = (u, v)$ with predicted logit z_e and observed state $a_e \in \{0, 1\}$, the Bernoulli loss $\mathrm{softplus}(z_e) - a_e z_e$ changes with a_e even when the evidence z_e is fixed. A pair MLP observes only geometry and roles, so its probability cannot change and it ranks by response at chance level (Fig. 8b). A query-masked neighbourhood model has an additional route: removing the queried link before computing the degree d_u of an endpoint yields $d_u - a_e$, so a switch changes the masked degree at the four target pairs although both graphs share the same degree sequence, an effect related to the target-link exclusion studied in [23]. Ranked by absolute change in probability, the masked GNN recovers 87.5% of the switched pairs among its top four (Fig. 8c). Holding its degree inputs fixed reduces this to 33.7%, and refitting with mean neighbourhood aggregation and no degree inputs on the same four GlaS folds, which reaches 95.7–98.2% development AUROC on normal links, reduces it to 46.6%, 40.9 points lower [32.8, 49.8]. Both interventions indicate a degree pathway, so recovering known graph edits is not evidence of validity.

8. Conclusion

We asked how much inspection effort a detector that catches every fault still imposes, and what in its training determines that effort. Matched valid and invalid edits answer both questions. The released composition branch detects every fault and ranks its targets well, yet at high recall it directs the inspector to 11 of 13 untouched compartments, a cost that image AUROC and top-k rates do not reveal and region overlap only ranks. The cost is set by the training target: marking every part of the inserted class suits interchangeable parts, but for parts tied to locations it teaches the network that one extra pin makes the whole tray anomalous, and marking only the pasted part reduces the cost to below half a compartment.

These findings have a clear scope. They concern one registered tray and synthetic composites rather than physical rearrangements; the top-2 ranking and sham specificity depend on the seed; and because the rule-aware class-wide controls fail the competence criterion, the mechanism rests on the continuation of the original recipe, where one seed passes. Natural next steps are physical rearrangements of real trays, retraining the full recipe with footprint targets, and recall–cost evaluation on further rules where parts are tied to locations, to make inspection cost a standard criterion.

LLM Usage Statement

OpenAI Codex was used to implement and run the experiments, check code and data provenance, make figures and tables from saved outputs, and draft text. Anthropic Claude was used to edit the text and redraw figures from the recorded outputs. Tray and network illustrations in Figs. 1–2 were generated with BioRender; labels were typeset separately and quantitative panels plot recorded results. The authors take full responsibility.

References

[1] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. 2

[2] Stephen Bates, Emmanuel Candès, Lihua Lei, Yaniv Romano, and Matteo Sesia. Testing for outliers with conformal p-values. *The Annals of Statistics*, 51(1):149–178, 2023. 2, 3

[3] Kilian Batzner, Lars Heckler, and Rebecca König. EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 128–138, 2024. 1, 2

[4] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9592–9600, 2019. 1, 2

[5] Paul Bergmann, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger. Beyond dents and scratches: Logical constraints in unsupervised anomaly detection and localization. *International Journal of Computer Vision*, 130 (4):947–969, 2022. 1, 2, 5, 8

[6] Matic Fučka, Vitjan Zavrtanik, and Danijel Skočaj. SALAD – semantics-aware logical anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 21843–21852, 2025. 1, 2, 4, 5, 8

[7] Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, Nitish Gupta, Hannaneh Hajishirzi, Gabriel Ilharco, Daniel Khashabi, Kevin Lin, Jiangming Liu, Nelson F. Liu, Phoebe Mulcaire, Qiang Ning, Sameer Singh, Noah A. Smith, Sanjay Subramanian, Reut Tsarfaty, Eric Wallace, Ally Zhang, and Ben Zhou. Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1307–1323, 2020. 2

[8] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. 2

[9] Chenrui Ji, Jiafan Lu, Zilu Luo, Feng Wan, Qiuhui Chen, and Weimin Zhong. SGDB: Semantic-geometric decoupling and evidence binding for logical anomaly detection. *arXiv preprint arXiv:2610.34897*, 2026. 2

[10] Divyansh Kaushik, Eduard Hovy, and Zachary C. Lipton. Learning the difference that makes a difference with counterfactually-augmented data. In *International Conference on Learning Representations*, 2020. 2

[11] Chun-Liang Li, Kihyuk Sohn, Jinsung Yoon, and Tomas Pfister. CutPaste: Self-supervised learning for anomaly detection and localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9664–9674, 2021. 2

[12] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017. 4

[13] Tongkun Liu, Bing Li, Xiao Du, Bingke Jiang, Xiao Jin, Liuyi Jin, and Zhuo Zhao. Component-aware anomaly detection framework for adjustable and logical industrial visual inspection. *Advanced Engineering Informatics*, 58:102161, 2023. 2, 5

[14] Danielle F. Pace, Hannah T. M. Contreras, Jennifer Romanowicz, Shruti Ghelani, Imon Rahaman, Yue Zhang, Patricia Gao, Mohammad Imrul Jubair, Tom Yeh, Polina Golland, Tal Geva, Sunil Ghelani, Andrew J. Powell, and Mehdi Hedjazi Moghari. HVSMR-2.0: A 3D cardiovascular MR dataset for whole-heart segmentation in congenital heart disease. *Scientific Data*, 11:721, 2024. 5

[15] Yun Peng, Xiao Lin, Nachuan Ma, Jiayuan Du, Chuangwei Liu, Chengju Liu, and Qijun Chen. SAM-LAD: Segment anything model meets zero-shot logic anomaly detection. *Knowledge-Based Systems*, 314:113176, 2025. 2

[16] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14318–14328, 2022. 1, 2

[17] Korsuk Sirinukunwattana, Josien P. W. Pluim, Hao Chen, Xiaojuan Qi, Pheng-Ann Heng, Yun Bo Guo, Li Yang Wang, Bogdan J. Matuszewski, Elia Bruni, Urko Sanchez, Anton Böhm, Olaf Ronneberger, Bassem Ben Cheikh, Daniel Racoceanu, Philipp Kainz, Michael Pfeiffer, Martin Urschler, David R. J. Snead, and Nasir M. Rajpoot. Gland segmentation in colon histology images: The GlaS challenge contest. *Medical Image Analysis*, 35:489–502, 2017. 5

[18] Tristan Thrush, Ryan Jiang, Max Bartolo, Amanpreet Singh, Adina Williams, Douwe Kiela, and Candace Ross. Winoground: Probing vision and language models for visio-linguistic compositionality. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5238–5248, 2022. 2

[19] Vitjan Zavrtanik, Matej Kristan, and Danijel Skočaj. DRAEM – a discriminatively trained reconstruction embedding for surface anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8330–8339, 2021. 2

[20] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. In *Advances in Neural Information Processing Systems*, 2018. 2

[21] Ximiao Zhang, Min Xu, and Xiuzhuang Zhou. LogiCo: A unified framework for logical and structural anomaly detection. *arXiv preprint arXiv:2606.28688*, 2026. 2, 5

[22] Ying Zhao. LogicAL: Towards logical anomaly synthesis for unsupervised anomaly localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 4022–4031, 2024. 2

[23] Jing Zhu, Yuhang Zhou, Vassilis N. Ioannidis, Shengyi Qian, Wei Ai, Xiang Song, and Danai Koutra. Pitfalls in link prediction with graph neural networks: Understanding the impact of target-link inclusion and better practices. In *ACM International Conference on Web Search and Data Mining*, pages 994–1002, 2024. 2, 8